

state of practice in electrical validation is an integrated process of (1) sampling the system response for a few sample parts, (2) identifying operating conditions under which the electrical behaviour lies outside specification and (3) optimisation, re-design and tuning as necessary to correct the problem.

Unlike logic and compatibility validation, electrical validation must account for statistical variation of system performance and noise tolerance across different process corners. PRQ requires the average defect to be low, typically less than 50 parts per million.

Speed path validation. The objective of speed path validation is to identify frequency-limiting design paths in the hardware. Because of variations, switching performance of different transistors in the design varies. This leads to data being propagated at different rates along different circuit paths. Speed at which the circuit can perform is ultimately constrained by the limitations of the slowest (in terms of data propagation speed) path in the design. Identifying such slow paths is therefore crucial in optimising design performance.

Speed path analysis includes identification of a potentially slow transistor, among millions or billions of these in a design, responsible for the speed path and the execution cycle (over a test, potentially millions of cycles long) that causes a slow transition.

Speed path debug makes use of a number of technologies including specialised testers, shmoo (2D plot) of chip failure pattern over voltage and frequency axes, DfD instrumentation available for observability, as well as laser-assisted observation of design internals and techniques for stretching and shrinking clock periods. More recently, analysis techniques based on formal methods have been successfully used for speed path identification.

Despite these latest developments, a significant ingenuity is necessary to isolate frequency-limiting paths for modern designs. Obviously, the above list of activities is not exhaustive. In addition to the above, validation covers behaviour of the system under extreme temperatures, physical stress and the like. Even the categories themselves are not cast in stone.

Post-silicon validation in practice

typically involves close collaboration between validators of different areas. For example, with increasingly tightening hardware/software integration in modern SoC designs, the boundary between basic hardware logic validation and compatibility validation with software has become blurred. In many cases, it is impossible to validate the hardware without also considering (at least) the firmware running on different IP cores. Indeed, post-silicon functional validation today often refers to the union of logic and compatibility validation.

Observability and controllability limitations

Limitations in observability and controllability constitute one of the key factors that distinguish validation based on a silicon artefact from pre-silicon activities. The problem arises because it is not possible to observe or control the billions of internal signals of the design during silicon execution. In order to observe a signal, its value must be routed to an observation point, such as an external pin or internal memory (for instance, trace buffer).

Consequently, the amount of observation that can be performed is limited by the number of pins or by the amount of memory dedicated for debug observability. Similarly, the amount of controllability depends on the number of configuration options defined by the architecture.

Both observability and controllability must be accounted for during the designing of the chip, since the hardware needs to be in place to route appropriate design signals to an observation point or to configure the system with specific controls.

On the other hand, during design, one obviously does not know what kind of design bugs may show up during post-silicon validation and what signals would be profitable to observe to debug these.

The current state of industrial practice primarily depends on designer experiences to identify observability. Any missing observability is typically only discovered at post-silicon—in the form of failure to root cause a given failure. Fixing observability at that time would require a new silicon spin, which is typically impractical.

Streamlining observability and error sequentiality

Traditional software or (pre-silicon) hardware debugging tends to work by sequentially finding and fixing bugs. We find a bug, fix it and then go on to finding the next bug. Unfortunately, this natural mental model of debugging breaks down for post-silicon. In particular, fixing a hardware bug found during post-silicon would require a new stepping.

Consequently, when a bug is discovered—even before the root cause for the bug is identified—one must find a way to workaround the bug to continue the post-silicon validation and debug process. Finding such workarounds is a challenging and creative process. On one hand, the workaround must eliminate the effect of the bug; on the other, it must not mask other bugs from being discovered.

Debugging in the presence of noise

A consequence of the fact that we are using actual silicon as the validation vehicle is that we must account for factors arising from physical reality in functional debug, that is, effects of temperature, electrical noise and others. A key challenge in post-silicon validation is to consequently find a recipe (for example, via tuning of different physical, functional and non-functional parameters) to make a bug reproducible.

On the other hand, the notion of reproducibility in post-silicon is somewhat weaker than in pre-silicon validation. Since post-silicon validation is fast, an error that reliably appears once in a few executions (even if not 100 per cent of the time) is still considered reproducible for post-silicon. Nevertheless, given the large space of parameters, ensuring reproducibility to the point that one can use it to analyse and diagnose the error is a significant challenge.

Security and power management challenge

Modern SoC designs incorporate highly-sophisticated architectures to support aggressive energy and security requirements. These architectures are typically defined independently by